

## §6: Softwaretests

Definition · Arten · Testprinzipien · Coverage · Pairwise

### 1 Was ist ein Softwaretest?

#### Definition 6.1 — Softwaretest (ANSI/IEEE 610.12-1990)

Ein **Softwaretest** ist der Prozess des Betriebens eines Systems oder einer Komponente unter festgelegten Bedingungen, das Beobachten oder Aufzeichnen der Ergebnisse und das Bewerten bestimmter Aspekte des Systems oder der Komponente.

**Vereinfacht (dieser Kurs):** Tests sind Programme, deren Ziel es ist, Fehler in anderen Programmen zu finden.

#### Definition 6.2 — Vier Fälle beim Testen

Ein Test versucht, ein Gegenbeispiel zur Hypothese „*der Algorithmus ist korrekt*“ zu finden. Es gibt vier Fälle:

| Alg. | Test | Bedeutung                                                    |
|------|------|--------------------------------------------------------------|
| +    | +    | Algorithmus ist richtig, kein Gegenbeispiel gefunden         |
| –    | –    | Alg. ist falsch, und der Test erkennt den Fehler             |
| +    | –    | Bug im Test (Gegenbeispiel, obwohl Alg. richtig ist)         |
| –    | +    | Test hat versagt, da er den Fehler im Alg. nicht erkannt hat |

### 2 Arten von Tests

#### 2.1 Black-box, Gray-box, White-box

##### Definition 6.3 — Black-box Test

Die **Spezifikation**, aber *nicht* die Implementierung ist bekannt. Alle Tests sowie Eingaben und Referenzergebnisse werden aus der Spezifikation abgeleitet.

**Beispiel:** `self.assertEqual(mysqrt(9), 3)` (aus der Spec:  $\sqrt{9} = 3$ )

##### Definition 6.4 — Gray-box Test

Die Implementierung ist bekannt, kann aber *nicht direkt modifiziert* werden. Testdaten werden aus der Implementierung hergeleitet (z. B. welcher Pfad durchlaufen wird).

**Beispiel:** `self.assertEqual(mysqrt(4), 2)` (deckt den Fall ab, dass die `while`-Schleife tatsächlich eintritt)

### Definition 6.5 — White-box Test

**Voller Zugriff** auf den Quellcode; Wissen wird gezielt eingesetzt, um Pfade, Schleifen und Bedingungen abzudecken. Techniken:

- **Assertions einbauen:** Invarianten direkt im Code prüfen (z.B. `assert(y!=0.0)` im Newton-Verfahren).
- **Fault Injection:** Absichtlich Bugs einbauen (automatisch deaktiviert nach dem Testen), um zu prüfen, ob die Testsuite mächtig genug ist.
- **Code Coverage Instrumentation:** Zusätzlicher Code stellt fest, welche Programmzeilen von keinem Test aufgerufen werden.

## 2.2 Unittest, Integrationstests, Systemtests

### Definition 6.6 — Testebenen

- **Unittest:** Überprüft die *kleinste isolierte Einheit* des Codes (Funktion/Methode).  $\Rightarrow$  Am wichtigsten für diesen Kurs und die Übungsblätter.
- **Integrationstest:** Prüft das *Zusammenspiel* und den Datenaustausch zwischen verschiedenen Modulen oder Komponenten.
- **Systemtest:** Validiert die *gesamte, vollständig integrierte Anwendung* gegen funktionale und technische Anforderungen aus Nutzersicht.

## 3 Häufige Fehlertypen

### Definition 6.7 — Off-by-one-Fehler

Ein **Off-by-one-Fehler** entsteht durch falsche Grenzwerte in Vergleichen ( $<$  statt  $\leq$  o. ä.).

**Test:** Kleinstmögliche Eingaben manuell nachvollziehen.

### Definition 6.8 — Integer-Überlauf

In C (nicht in Python): Der Wert einer Integer-Variable überläuft den Maximalwert und wechselt zyklisch. *Python-Integer sind beliebig lang — kein Überlauf.*

### Definition 6.9 — Float-Überlauf und Loss of Precision

- **Float-Überlauf:** Der maximale Float-Wert liegt bei ca.  $1,79 \times 10^{308}$ . Darüber: `OverflowError`.
- **Loss of Precision:** Gleitkommazahlen können nicht alle ganzen Zahlen exakt darstellen.  $a = 2.0^{53}$  ist die größte mit float exakt darstellbare ganze Zahl. Danach gilt:  $a + 1 = a$ .
- **Rundungsfehler:** z.B. `math.sqrt(0.5)**2 = 0.5000000000000001`  $\Rightarrow$  `assertAlmostEqual` statt `assertEqual` verwenden.

### Beispiel 6.10 — Quadratwurzel mit Newton-Verfahren (mysqrt)

$y = \sqrt{x}$  ist die Nullstelle von  $f(y) = x - y^2 = 0$ . Newton-Iteration:  $y^{(t+1)} = \frac{y^{(t)} + x/y^{(t)}}{2}$

```
def mysqrt(x):  
    if (x < 0):
```

```

        raise ValueError("mysqrt: Negative numbers are not allowed!")
    y = x / 2
    while y**2 != x:
        y = (y + x/y) / 2
    return y

```

**Problem:** Wegen Rundungsfehlern terminiert die Schleife bei nicht-rationalen Wurzeln möglicherweise nie. Für  $x = 4$  terminiert sie, da  $2^2 = 4$  exakt darstellbar ist.

## 4 Prinzipien zur Generierung von Testdaten

### 4.1 Referenzergebnisse

#### Definition 6.11 — Methoden zur Referenzerzeugung

- **Einfache Eingaben:** Manuell berechenbares Ergebnis. `assertEqual(mysqrt(9), 3)`
- **Alternatives Verfahren:** Langsamere, aber vertrauenswürdige Implementierung. `assertAlmostEqual(mysqrt(0.5), math.sqrt(0.5), 15)`
- **Indirektes Prüfen:** Eigenschaft des Ergebnisses testen statt Ergebnis selbst. `assertAlmostEqual(mysqrt(0.5)**2, 0.5, 15)`
- **Randwerte:** Grenzen des Definitionsbereichs explizit testen. `assertEqual(mysqrt(0), 0)`

### 4.2 Regressionstests und Failure Reproduction

#### Definition 6.12 — Regressionstests

**Regressionstests** verhindern, dass bereits behobene Fehler durch spätere Code-Änderungen wieder auftreten. **Profi-Ansatz:**

1. Test *vor* dem Programm schreiben (Test-Driven Development).
2. Nach erfolgreichem Test: Test in die Testsuite *dauerhaft* aufnehmen.
3. Bei jeder Programmänderung: *alle* Tests laufen lassen.

**Failure Reproduction:** Beim Auftreten eines Bugs zuerst einen Test schreiben, der den Bug reproduziert — dann beheben, dann Test in Testsuite aufnehmen.

### 4.3 Äquivalenzklassen (Equivalence Partitioning)

#### Definition 6.13 — Prinzip der äquivalenten Eingaben

**Problem:** Die Menge möglicher Eingaben ist zu groß, um sie vollständig zu testen.

**Lösung:** Eingaben in Gruppen  $g_i$ ,  $i \in [n]$  aufteilen, die sich algorithmisch gleich verhalten. Dann:

- Für jedes  $i$ : mindestens 1 Test aus  $g_i$
- Randwerte zwischen  $g_i$  und  $g_j$  ( $i \neq j$ ) explizit testen

**Beispiel mysqrt:**

| Gruppe | Beschreibung                           | Testbeispiel                                            |
|--------|----------------------------------------|---------------------------------------------------------|
| $g_1$  | Negative Zahlen                        | <code>assertRaises(ValueError, mysqrt, -1)</code>       |
| $g_2$  | Ganze Zahlen                           | <code>assertEqual(mysqrt(9), 3)</code>                  |
| $g_3$  | Spezialfall $x = 1$                    | <code>assertEqual(mysqrt(1), 1)</code>                  |
| $g_4$  | Reelle Zahlen                          | <code>assertAlmostEqual(mysqrt(0.5)**2, 0.5, 15)</code> |
| $g_5$  | While-loop nicht ausgeführt (Grey-box) | <code>assertEqual(mysqrt(4), 2)</code>                  |
| Rand   | Grenze $g_1/g_2$                       | <code>assertEqual(mysqrt(0), 0)</code>                  |

#### 4.4 Code Coverage

##### Definition 6.14 — Code-Coverage-Stufen (Grey-box)

- **Function coverage:** Jede Funktion muss mind. einmal aufgerufen werden.
- **Statement coverage:** Jede Programmzeile muss mind. einmal ausgeführt werden.  
*Beispiel:* `assertRaises(ValueError, mysqrt, -1)` deckt die `raise`-Zeile ab.
- **Condition coverage:** Jede Bedingung muss mind. einmal `True` und einmal `False` ergeben (gilt für `if`, `while`, `for`).  
*Beispiel:* `assertEqual(mysqrt(4), 2) → while-Bedingung wird True, dann False.`
- **Path coverage:** Jede Kombination aller Wahrheitswerte aller Bedingungen.  
*Im Allgemeinen unerreichbar:* # Pfade  $\in O(2^k)$ ,  $k = \#$  if/while-Bedingungen.

#### 4.5 Pairwise Coverage

##### Definition 6.15 — Prinzip der vollständigen Paarung (Pairwise Coverage)

**Aufgabe:** Eine Funktion  $f(p_1, \dots, p_n)$  mit Parametern  $p_i \in P_i$ ,  $|P_i| < \infty$  testen.

**Naive Vollständigkeit:**  $\prod_{i=1}^n |P_i|$  Eingaben — bei  $|P_i| = k$  sind das  $k^n$  Tests.

**Pairwise-Idee:** Statt aller Kombinationen nur alle *Paare* abdecken:

$$\{(p_i, p_j) : p_i \in P_i, p_j \in P_j\} \quad \text{für alle } i \neq j.$$

**Theorem:** Die Anzahl benötigter Testeingaben liegt in  $O(k^2)$ , wobei  $k = \arg \max_{i \in [n]} |P_i|$  die maximale Parameterwertanzahl ist.

Reduzierung von  $k^n$  auf  $O(k^2)$  — enorme Einsparung bei großem  $n$ .

##### Beispiel 6.16 — unittest-Klasse für mysqrt

```
import unittest

class mysqrtTests(unittest.TestCase):

    def test_mysqrt(self):
        self.assertRaises(ValueError, mysqrt, -1)      # g1: Negative
                                                       Zahlen
        self.assertEqual(mysqrt(9), 3)                 # g2: Ganze Zahlen
        self.assertEqual(mysqrt(1), 1)                 # g3: Spezialfall
        self.assertAlmostEqual(mysqrt(0.5)**2, 0.5, 15) # g4: Reelle Zahlen
        self.assertEqual(mysqrt(4), 2)                 # g5: while-Schleife
```

```

        self.assertEqual(mysqrt(0), 0)                                # Randwert

if __name__ == "__main__":
    unittest.main()

```

### Achtung

**Floats niemals mit assertEquals vergleichen!** Da Gleitkommaoperationen rundungsbehaftet sind, ist `assertEquals(mysqrt(0.5)**2, 0.5)` fast immer falsch. Stattdessen: `assertAlmostEqual(..., places=15)`

## 5 Vergleich: Testarten

|               |      | Black-box                 | Gray-box                  | White-box                   |
|---------------|------|---------------------------|---------------------------|-----------------------------|
| Quellcode     | be-  | Nein                      | Ja (nur lesen)            | Ja (Zugriff)                |
| kannt?        |      |                           |                           |                             |
| Testdaten aus |      | Spezifikation             | Implementierung           | Pfade, Bedingungen          |
| Eingriff      | mög- | Nein                      | Nein                      | Ja (Assertions, Fault       |
| lich?         |      |                           |                           | Injection)                  |
| Beispiel      |      | <code>mysqrt(9)==3</code> | <code>mysqrt(4)==2</code> | <code>assert(y!=0.0)</code> |

|                  | Unittest                 | Integrationstest  | Systemtest   |
|------------------|--------------------------|-------------------|--------------|
| Granularität     | Einzelne Funktion        | Modul-Interaktion | Gesamtsystem |
| Prüft            | Korrektheit der Funktion | Schnittstellen    | Nutzersicht  |
| Priorität (Kurs) | <b>Höchste</b>           | Mittel            | Niedrig      |

## Prüfungsscheckliste — §6 Softwaretests

---

- |                          |                                                                                                                                                              |                       |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <input type="checkbox"/> | Definition Softwaretest (ANSI/IEEE) angeben und vereinfachte Version erklären.                                                                               | → <i>Def. 6.1</i>     |
| <input type="checkbox"/> | Die vier Fälle (Alg./Test: +/+, -/-, +/-, -/+) beschreiben und ihre Bedeutung erläutern.                                                                     | → <i>Def. 6.2</i>     |
| <input type="checkbox"/> | Black-box-, Gray-box- und White-box-Tests unterscheiden und je ein Beispiel angeben.                                                                         | → <i>Def. 6.3–6.5</i> |
| <input type="checkbox"/> | Unittest, Integrationstest und Systemtest abgrenzen; wissen, welcher im Kurs am wichtigsten ist.                                                             | → <i>Def. 6.6</i>     |
| <input type="checkbox"/> | Off-by-one, Integer-Überlauf, Float-Überlauf und Loss of Precision erklären und je ein Beispiel geben.                                                       | → <i>Def. 6.7–6.9</i> |
| <input type="checkbox"/> | Warum <code>assertAlmostEqual</code> statt <code>assertEqual</code> bei Floats? Begründung mit Rundungsfehlern.                                              | → <i>Warn.</i>        |
| <input type="checkbox"/> | Drei Methoden zur Referenzerzeugung nennen: einfache Eingaben, alternatives Verfahren, indirektes Prüfen.                                                    | → <i>Def. 6.11</i>    |
| <input type="checkbox"/> | Regressionstests: Anfänger- vs. Profi-Vorgehen gegenüberstellen; Failure Reproduction erklären.                                                              | → <i>Def. 6.12</i>    |
| <input type="checkbox"/> | Äquivalenzklassen für <code>mysqrt</code> aufstellen (mind. 5 Gruppen) und Randwerte benennen.                                                               | → <i>Def. 6.13</i>    |
| <input type="checkbox"/> | Function-, Statement-, Condition- und Path-Coverage definieren und Beispiele für jede Stufe angeben.                                                         | → <i>Def. 6.14</i>    |
| <input type="checkbox"/> | Pairwise Coverage erklären: Problem ( $k^n$ Eingaben), Idee (alle Paare), Theorem ( $O(k^2)$ ).                                                              | → <i>Def. 6.15</i>    |
| <input type="checkbox"/> | Eine vollständige unittest-Klasse für <code>mysqrt</code> schreiben: <code>assertRaises</code> , <code>assertEqual</code> , <code>assertAlmostEqual</code> . | → <i>Bsp. 6.16</i>    |